Introduction to Algorithms and Data Structures

week 3.2: Recursion part 2

SAXION UNIVERSITY OF APPLIED SCIENCES

Agenda

- **Discuss work last lesson**
- Recap
- Iterative versus recursion
- Workshop
- Homework

- Where you able to do the exercises?
  1. Yes! All exercises are done!
  2. No.., Because I have these questions
  3. I haven't prioritized this subject enough at the moment to do the assignments

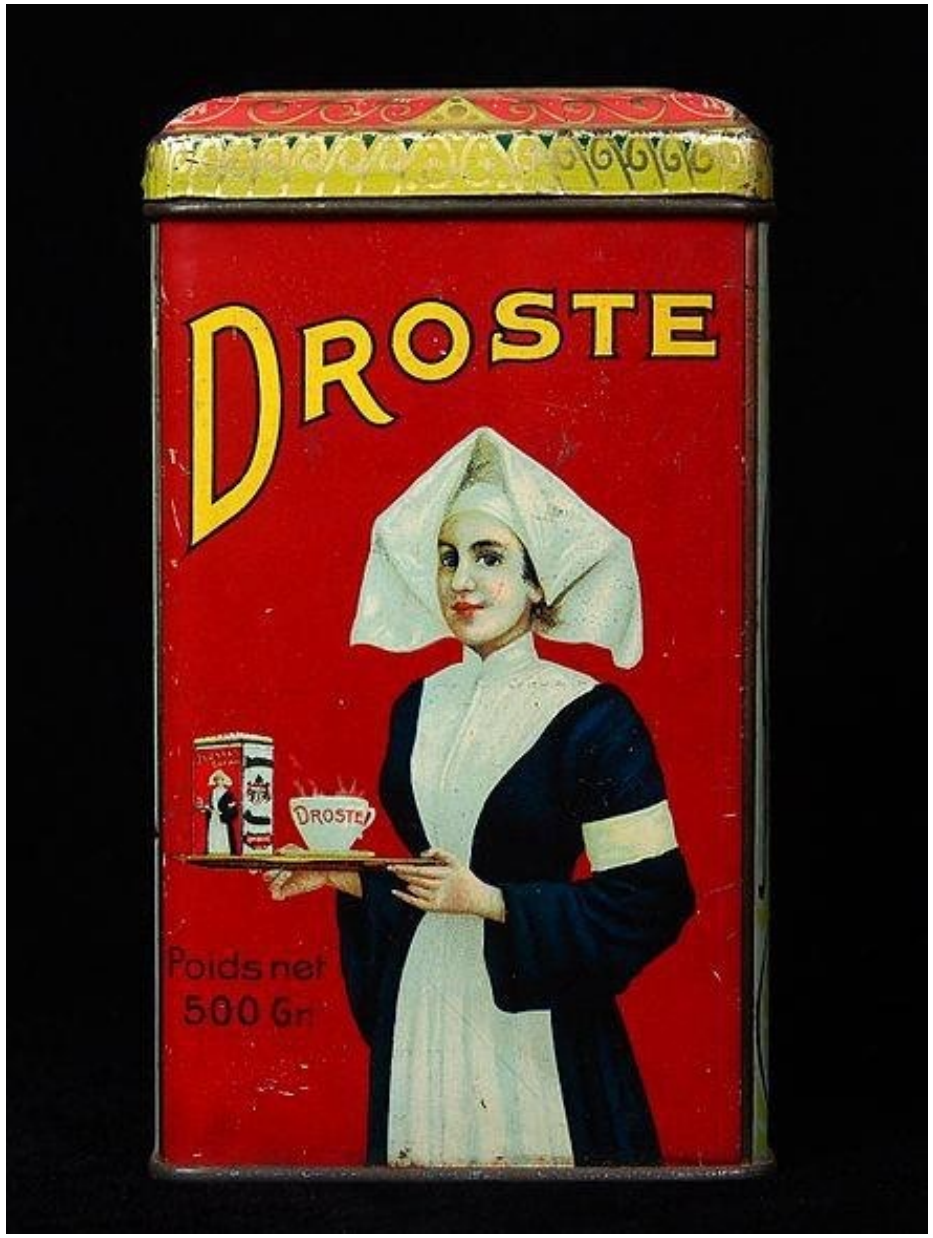
Recursion

- **Discuss work last lesson**
- **Recap**
- Iterative versus recursion
- Workshop
- Homework

Recap: What is recursion?

- Recursion is a technique where a function solves a problem by:
  - **breaking it down** into smaller, similar sub-problems, by **calling itself** (recurse) to solve these sub-problems,
  - which **stops** when reaching a **base case***.

***Base case:** a problem small enough that it can be solved directly without further subdivision



Recap: Always try to make the problem smaller!

- Base cases solve the simplest problems, providing endpoints.
- Break down the problem into smaller, alike versions until reaching these base cases.
- Employ recursion when problems can be methodically simplified into smaller, identical forms, leading to resolution through repeated reduction.

- *Or simpler: Use a recursive solution for problems that can be reduced to the same problem but smaller! Until a base case is reached*

ReversedString:

reversedString("abcd");

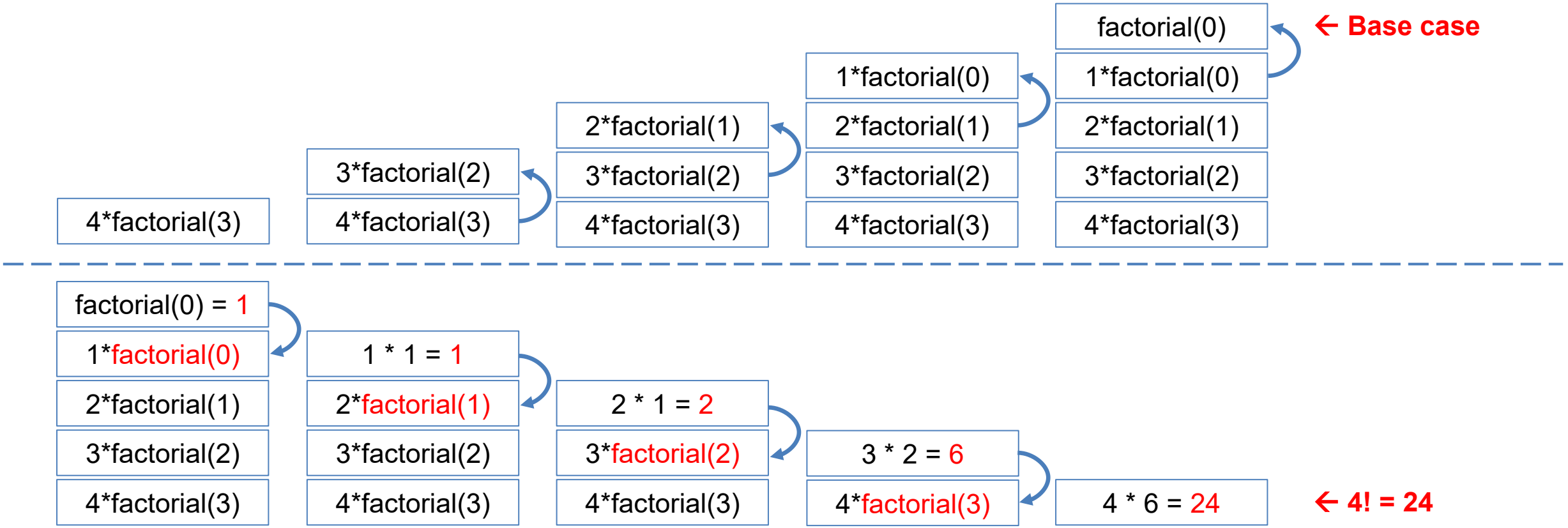
```
public string reversedString(String word) {
        // base case 1
        if (word.length <= 1) {
                return word;
        }

        // base case 2
        if (word.length == 2) {
                return "" + word.charAt(1) + word.charAt(0); //convert to string
        }

        // recursion part (if length > 2)
        char lastChar = word.charAt(word.length()-1);
        String firstPart = word.substring(0,word.length()-1);

        return lastChar + reversedString(firstPart);
}
```

What's strange about this solution?

Recursion

- **Discuss work last lesson**
- **Recap**
- **Iterative versus recursion**
- Workshop
- Homework

•Calculate 4!


factorial(0)
← Base case
1*factorial(0)
1*factorial(0)
2*factorial(1)
2*factorial(1)
2*factorial(1)
3*factorial(2)
3*factorial(2)
3*factorial(2)
3*factorial(2)
4*factorial(3)
4*factorial(3)
4*factorial(3)
4*factorial(3)
4*factorial(3)
factorial(0) = 1
1*factorial(0)
1 * 1 = 1
2*factorial(1)
2*factorial(1)
2 * 1 = 2
3*factorial(2)
3*factorial(2)
3*factorial(2)
3 * 2 = 6
4*factorial(3)
4*factorial(3)
4*factorial(3)
4 * 6 = 24
← 4! = 24

Recursive sum method

```
public static long sumRecursive(long n){
    long sum = n;
    if(n==1) return 1;
    else return sum += sumRecursive(n:n-1);

}
```

Iterative sum method

```
public static long sumIterative(long n){
    long sum = 0;
    while(n>0){
        sum += n;
        n--;
    }
    return sum;
}
```

Of the two methods…

- Which is the more elegant one?
- Which is the more efficient one?

In general, …

- Does efficiency matter if you have a fast computer?

```
Enter in a positive integer, between 1 and 10000: 10000
Sum of all integers up to 10000: 50005000
Recursive method: 10.1619ms

Sum of all integers up to 10000: 50005000
Iterative method: 0.3953ms
```

Sidenote…

- In Java you can get the current value of the system timer in nanoseconds by calling the method **System.nanoTime();**
- It will return a long value back.
- To calculate milliseconds, divide the nanoseconds by $1000000 = 10^6$
- Or use **System.currentTimeMillis();**

```
Enter in a positive integer, between 1 and 10000: 10000
Sum of all integers up to 10000: 50005000
Recursive method: 10.1619ms

Sum of all integers up to 10000: 50005000
Iterative method: 0.3953ms
```

Question:

- How did we measure the execution duration of a method?

More information on System class:

edu.nl/qpjwn

Conclusion

- Iteration is more efficient than recursion
- Recursion can lead to stack overflow exceptions
- However, some problems are easier to solve if you approach them recursively. We will elaborate on that in the next slides.

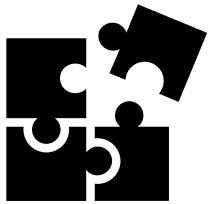


Image by: smial, CC BY-SA 2.0 DE, via Wikimedia Commons

Recursion

- **Discuss work last lesson**
- **Recap**
- **Iterative versus recursion**
- **Workshop**
- Homework

Workshop:

Do exercise: 4_stepsToSolve from **recursion**

You can find this on Brightspace > week3

Workshop:

Do exercise: 6_strangeLanguage from **recursion**

You can find this on Brightspace > week3

Recursion

- Discuss work last lesson
- Recap
- Iterative versus recursion
- Workshop
- Homework

SAXION UNIVERSITY OF APPLIED SCIENCES